

Cloud Computing & Security

MCQ Concept Mastery Notes

Explained for Absolute Beginners

Part A — Cloud Computing Concepts

Q1: SLA & Availability Calculation Q2: Multi-core Power Consumption
Q3: Hybrid Cloud Economics (CapEx vs OpEx)
Q4: Elastic Scalability Q5: Virtualization
Q6: MapReduce & Data Locality Q7: HDFS Block Size
Q8: Containers vs Virtual Machines
Q9: Energy-aware Consolidation Q10: Load Balancing & Hotspots

Part B — Cloud Security Concepts

Q1: Replay Attack & Authenticity Q2: Phishing & Confidentiality
Q3: Passive Attacks Q4: Slow HTTP Attack & Disruption
Q5: Penetration Testing
Q6: Non-repudiation & Digital Signatures
Q7: Authentication vs Authorization
Q8: PaaS Q9: Data Location Q10: Vendor Lock-in

Contents

1	Part A — Cloud Computing MCQ Concepts	3
1.1	Q1 — SLA Availability Calculation	3
1.1.1	The Scenario	3
1.1.2	The Availability Formula	3
1.2	Q2 — Multi-core Power Consumption	5
1.2.1	What Happens When More Cores Are Activated?	5
1.3	Q3 — Hybrid Cloud Economics: CapEx vs OpEx	6
1.3.1	What Changes in a Hybrid Cloud?	6
1.4	Q4 — Elastic Scalability	7
1.4.1	What is Elastic Scalability?	7
1.5	Q5 — Virtualization: Multiple OS on One Machine	8
1.5.1	How Virtualization Works	8
1.6	Q6 — MapReduce and Data Locality	10
1.6.1	The Key Problem: Data is Already Spread Across Machines	10
1.7	Q7 — HDFS Block Size and Performance	11
1.7.1	What Happens When Block Size Increases?	11
1.7.2	Why Fewer Mappers = Less Overhead?	11
1.8	Q8 — Containers vs Virtual Machines	12
1.8.1	Virtual Machines (VMs) in Detail	12
1.8.2	Containers in Detail	12
1.9	Q9 — Energy-Aware Server Consolidation	14
1.9.1	The Scenario	14
1.9.2	What Are the Two Effects?	14
1.10	Q10 — Avoiding Hotspots: Load Balancing and VM Migration	15
1.10.1	Two Techniques to Avoid Hotspots	15
2	Part B — Cloud Security MCQ Concepts	17
2.1	Q1 — Replay Attack and Authenticity	17
2.1.1	What Security Property is Violated?	17
2.2	Q2 — Phishing Attack and Confidentiality	18
2.3	Q3 — Passive Attack: Traffic Analysis	18
2.3.1	Traffic Analysis — What is It?	18
2.4	Q4 — Slow HTTP Attack and Disruption	20
2.4.1	The Slow HTTP Attack	20
2.5	Q5 — Penetration Testing	21
2.5.1	What Makes Penetration Testing Unique?	21
2.6	Q6 — Non-repudiation and Digital Signatures	22
2.6.1	How is Non-repudiation Achieved?	22
2.7	Q7 — Authentication vs Authorization	23
2.7.1	Authentication — Proving Your Identity	23
2.7.2	Authorization — Deciding What You Can Do	23
2.8	Q8 — PaaS: The Middle Ground	25
2.8.1	Why PaaS is the Right Choice for Developers	25
2.8.2	What PaaS Specifically Provides	25
2.9	Q9 — Data Location / Data Residency	26
2.9.1	Why This is a Cloud Challenge	26

2.9.2	Different Countries, Different Rules	26
2.10	Q10 — Vendor Lock-in	27
2.10.1	How Vendor Lock-in Happens	27
3	Master Revision Cheat Sheet — All 20 Questions	29
3.1	Part A: Cloud Computing	29
3.2	Part B: Cloud Security	30

Part A — Cloud Computing MCQ Concepts

1.1 Q1 — SLA Availability Calculation

What is SLA and Availability?

SLA (Service Level Agreement) is a contract between a cloud provider and a customer that guarantees a minimum level of service. One of the most important guarantees is **availability** — the percentage of time the system must be working.

99% availability means: the system can be **down (broken/offline)** for at most **1%** of the total scheduled time. If downtime exceeds 1%, the SLA is **violated**.

► The Scenario

- Application runs **Y hours per day**
- For **20 days**
- Total scheduled time = $20 \times Y$ hours
- Actual downtime = **4 hours**

► The Availability Formula

$$\text{Availability} = \frac{\text{Total Time} - \text{Downtime}}{\text{Total Time}} = 1 - \frac{4}{20Y}$$

For SLA to be **satisfied**, availability must be at least 99%, i.e., ≥ 0.99 .

When SLA is **violated**, availability < 0.99 :

$$1 - \frac{4}{20Y} < 0.99 \implies \frac{4}{20Y} > 0.01 \implies 20Y < \frac{4}{0.01} = 400 \implies Y < 20$$

Correct Answer: $Y < 20$

If the application runs for **fewer than 20 hours per day**, then the 4 hours of downtime takes up more than 1% of the total time, causing SLA violation.

If it runs for 24 hours a day (full day), downtime is a much smaller fraction and SLA is satisfied.

Real-Life Analogy

Imagine you hire a worker for a certain number of hours. If the worker is absent for 4 hours:

- Hired for **8 hours** → absent 50% (very bad — SLA violated)
- Hired for **24 hours** → absent 16.7% (still bad here)
- Hired for **400+ hours** → absent only 1% (SLA met)

The fewer hours you schedule, the bigger the impact of the same downtime.

Quick Tip / Memory Trick

Key Intuition: Less runtime $\rightarrow$ same downtime becomes a larger fraction $\rightarrow$ SLA breaks.

Solve: Find when downtime fraction $> 1\% \rightarrow$ That gives you $Y < 20$.

1.2 Q2 — Multi-core Power Consumption

What Are CPU Cores?

A **CPU core** is a single processing unit inside your processor. Modern processors have **multiple cores** (e.g., 4-core, 8-core) so they can do many tasks at the same time. Think of cores like **workers in a factory**. More workers means more work gets done simultaneously — but they all need electricity while working.

► What Happens When More Cores Are Activated?

Real-Life Example: Your Laptop

- **Idle (doing nothing)** → Very low power, most cores sleeping
- **Open Chrome + VS Code** → More cores wake up, power increases
- **Run heavy code on all cores** → Maximum power usage

Every time you wake up more cores and put them to work, the electricity drawn from the battery **immediately increases** at that moment.

Correct Answer: Activating more cores increases instantaneous power draw

Instantaneous power = the electricity being used at this exact moment.

When more cores are running tasks simultaneously, more circuits are switching and consuming energy — so the power draw at that instant goes up.

Note: Over a longer period, if multi-core processing finishes a task faster, total energy might be similar. But the question is about **instantaneous** (right now) power — which definitely increases.

Why Other Options Are Wrong

- “Power always decreases” → Wrong. More active cores = more draw.
- “Relationship is strictly linear” → Not exactly; depends on workload, clock speed, etc.
- “Power usage stays the same” → Incorrect; idle cores use very little, active ones use much more.

Quick Tip / Memory Trick

Remember: **More cores ON = more power NOW**. Like switching on more light bulbs — each bulb on draws more electricity immediately.

1.3 Q3 — Hybrid Cloud Economics: CapEx vs OpEx

Two Types of Spending

- **CapEx (Capital Expenditure):** Money spent **upfront** to buy something permanently. You own the asset after purchase.
Example: Buying servers, purchasing hardware equipment, building a data centre.
- **OpEx (Operational Expenditure):** Money spent **regularly/continuously** for using a service. You pay as you use.
Example: Monthly AWS bill, paying for cloud storage, internet subscriptions.

► What Changes in a Hybrid Cloud?

Before hybrid cloud (fully on-premise):

- Company buys servers, racks, networking hardware → High CapEx
- Low OpEx (no cloud bills)

After moving to hybrid cloud (own + cloud):

- Company buys *fewer* physical servers → **CapEx decreases**
- Company now pays regular cloud bills → **OpEx increases**

Correct Answer: OpEx increases relative to CapEx

In a hybrid cloud model, the biggest financial shift is: you stop spending big money upfront on hardware and instead pay ongoing cloud service bills. The balance **shifts from CapEx to OpEx**.

Real-Life Analogy

- **Before:** Buying a car (CapEx) — big upfront payment, you own it forever
 - **After:** Using Uber/Ola (OpEx) — no big purchase, but you pay every ride
- In hybrid cloud: you stop “buying cars” and start “taking Uber more often.” Big upfront cost drops, but monthly bills rise.

Quick Tip / Memory Trick

Formula to remember:

Hybrid Cloud $\Rightarrow$ CapEx $\downarrow$ (buy less hardware) OpEx $\uparrow$ (pay cloud bills)

1.4 Q4 — Elastic Scalability

The Problem: Unpredictable Traffic

Imagine you run an e-commerce website like Flipkart.

- Normal day: 1,000 users online
- Big sale (Republic Day / Diwali): 10,00,000 users online

If you buy fixed servers for 10 lakh users → you waste money on normal days.

If you buy servers for 1,000 users → your site crashes on sale day.

► What is Elastic Scalability?

Elasticity means the system can **automatically stretch or shrink** based on demand:

- **Traffic increases** → Cloud automatically adds more servers
- **Traffic decreases** → Cloud automatically removes servers
- **You pay only for what you use** at any given moment

Food Delivery App Example

- **Lunchtime (12–2 PM):** High traffic → 100 servers running
- **Midnight:** Low traffic → 10 servers running
- **Savings:** You don't pay for 90 idle servers at midnight!

Correct Answer: Elastic Scalability improves cost efficiency for unpredictable workloads

By automatically adjusting resources to actual demand, elastic scalability ensures:

- No wasted money on idle servers
- No crashes due to under-provisioning
- Cost matches actual usage perfectly

Why Other Options Are Wrong

- **High availability** → About keeping the system running 24/7, not about cost
- **Redundancy** → About having backup systems, not about cost efficiency
- **Load balancing** → About distributing work evenly, not about auto-scaling cost

Quick Tip / Memory Trick

Elasticity = Pay only for what you actually use, right now. Like a water tap — you pay for water when it flows, not for the pipe's capacity.

1.5 Q5 — Virtualization: Multiple OS on One Machine

The Core Question

“How can multiple operating systems run simultaneously on a single physical machine?”

The answer is **Virtualization**.

► How Virtualization Works

A special software called a **Hypervisor** sits between the physical hardware and all the operating systems. It creates multiple **Virtual Machines (VMs)**, each acting like a completely separate real computer.

Your Own Laptop Example

- Physical laptop with Windows → This is your real machine
- Install Oracle VirtualBox (the Hypervisor)
- Inside VirtualBox, install Kali Linux
- Now both Windows AND Linux run on the same laptop simultaneously!
- Kali Linux thinks it's on a real computer — it doesn't even know it's virtual

Correct Answer: Virtualization

The hypervisor creates virtual CPUs, virtual RAM, and virtual disks for each VM. Each VM is completely isolated and runs its own OS independently. The physical hardware is shared, but each OS feels like it has its own dedicated machine.

Why Other Options Are Wrong

Why not Containerization?

Containers (like Docker) are *different* from virtualization:

- Containers **share the same OS kernel** — they don't run a separate OS
- Containers are lighter and faster, but they cannot run a completely different OS
- Example: A container on Linux cannot run Windows inside it
- Virtualization can — each VM has its own complete, independent OS

Feature	Virtualization (VM)	Containerization
Own OS?	Yes, each VM has full OS	No, shares host OS kernel
Isolation	Strong (like separate rooms)	Weaker (like partitions)
Weight	Heavy (GBs)	Lightweight (MBs)
Run different OS?	Yes	No
Example	VirtualBox, VMware	Docker, Kubernetes

Quick Tip / Memory Trick

Multiple OS on one machine = Virtualization. Hypervisor creates the magic. Containers only isolate apps, not entire operating systems.

1.6 Q6 — MapReduce and Data Locality

What is MapReduce?

MapReduce is a technique used in cloud systems (like Hadoop) to process **huge amounts of data** (think terabytes or petabytes) by splitting the work across many machines.

It has two main phases:

- **Map phase:** Break the data into small pieces and process each piece on a separate machine (called a **node**).
- **Reduce phase:** Combine all the results from the Map phase into a final answer.

► The Key Problem: Data is Already Spread Across Machines

In systems like HDFS (Hadoop Distributed File System), huge files are split into **blocks** and stored on different machines in the cluster. For example:

- Block 1 stored on Machine A
- Block 2 stored on Machine B
- Block 3 stored on Machine C

When MapReduce runs, it needs to process each block. You have two choices:

1. **Move the data** to the machine doing the computation → Slow! Network traffic is expensive.
2. **Move the computation** to the machine where the data already lives → Fast! No data movement needed.

Correct Answer: Schedule mappers on nodes that store the data (Data Locality)

Data Locality = run the processing task **on the same machine** where the data already is.

This avoids slow network transfers and dramatically improves performance.

Real-Life Analogy

Imagine you need to read 100 large books in a library:

- **Option 1:** Transport all books to your house, read there → Expensive, slow
- **Option 2:** Go to the library and read there → Fast, efficient

MapReduce uses Option 2: it sends the *task* to where the *data* already lives.

Quick Tip / Memory Trick

Data Locality Principle: Don't move data to the code. Move the code to the data. Scheduling mappers on the same node as their data = best MapReduce performance.

1.7 Q7 — HDFS Block Size and Performance

What is HDFS and Block Size?

HDFS (Hadoop Distributed File System) stores very large files by splitting them into fixed-size **blocks** and distributing these blocks across many machines in the cluster. Default block size in HDFS = **128 MB** (sometimes 256 MB or larger). Each block is processed by **one mapper** in the MapReduce Map phase.

► What Happens When Block Size Increases?

Let's use a concrete example with a **1 GB** file:

Block Size	Number of Blocks	Number of Mappers	Overhead
128 MB (default)	8 blocks	8 mappers	Higher
256 MB	4 blocks	4 mappers	Medium
512 MB (larger)	2 blocks	2 mappers	Lower

► Why Fewer Mappers = Less Overhead?

Each mapper has a **fixed overhead** cost:

- Starting the mapper task (JVM startup time)
- Scheduling the task through the cluster manager
- Tracking its progress
- Collecting its result

Real-Life Analogy

Imagine assigning homework to students:

- **100 tiny tasks:** Each student needs instructions, supervision, follow-up. Too much coordination overhead.
- **5 big tasks:** Much less coordination. More efficient.

Same logic applies to mappers in HDFS.

Correct Answer: Larger block size reduces number of mappers and reduces scheduling overhead

Increasing HDFS block size $\Rightarrow$ fewer blocks $\Rightarrow$ fewer mappers $\Rightarrow$ less task startup and scheduling overhead $\Rightarrow$ better efficiency for large files.

Quick Tip / Memory Trick

Remember: More blocks = more mappers = more overhead.

Bigger blocks = fewer mappers = less overhead = faster for large datasets.

1.8 Q8 — Containers vs Virtual Machines

The Core Difference

Both VMs and Containers allow you to run multiple isolated applications on one machine. But they do it in **very different ways** with very different trade-offs.

► Virtual Machines (VMs) in Detail

- Each VM runs its own **complete operating system**
- Hypervisor provides **strong isolation** — each VM is like a completely separate computer
- **Heavy:** Each VM might need 2–20 GB of disk space, significant RAM
- **Slower to start:** Booting a VM is like booting a real computer (minutes)
- Fewer VMs can run on the same hardware due to overhead

► Containers in Detail

- All containers **share the host OS kernel** — no separate OS per container
- **Lightweight:** A container might only be a few hundred MB
- **Fast to start:** Containers start in seconds (sometimes milliseconds)
- Many more containers can run on the same hardware (**higher density**)
- **Weaker isolation:** Since they share the OS kernel, a kernel vulnerability affects all containers

Correct Answer: Containers achieve higher density but provide weaker isolation than VMs

- **Higher density** = more containers can run on the same physical hardware compared to VMs
- **Weaker isolation** = sharing the OS kernel means less security separation between containers

Real-Life Analogy

- **VMs = Separate houses** — each family has their own completely independent building. Very private, but land usage is inefficient.
- **Containers = Apartments in one building** — all apartments share plumbing and electrical systems (the OS kernel). More efficient use of space (higher density), but a building-wide problem affects everyone.

Feature	Virtual Machine	Container
OS	Own complete OS	Shares host OS kernel
Isolation	Strong	Weaker
Size	GBs (heavy)	MBs (light)
Density	Lower (few per host)	Higher (many per host)
Startup time	Minutes	Seconds
Example	VirtualBox, VMware	Docker, Podman

1.9 Q9 — Energy-Aware Server Consolidation

What is Server Consolidation?

Server Consolidation is the process of taking workloads spread across many half-empty servers and moving them all onto **fewer servers that are more fully utilized**. The unused, now-empty servers are **switched off** to save electricity.

► The Scenario

- **Before:** 8 servers running, each at **50% CPU utilization**
- **After consolidation:** 5 servers running, each at **75% CPU utilization**
- 3 servers are switched off completely

► What Are the Two Effects?

Effect 1 — Power Savings:

- 3 fewer servers are running → **total power consumption decreases**
- Even though remaining servers work harder, the net electricity bill goes down

Effect 2 — Reduced Capacity / Higher SLA Risk:

- Servers now run at 75% instead of 50%
- Less **headroom** (spare capacity) available
- If there is a sudden spike in workload → servers get overwhelmed quickly
- This can lead to **SLA violations** (response time increases, availability drops)

Correct Answer: Total power decreases; SLA violation risk increases

Real-Life Analogy

Factory workers analogy:

Before: 8 workers doing moderate work (relaxed, spare energy)

After consolidation: 5 workers doing the same total work (busier)

- Salary bill (power) drops because you're paying fewer workers
- But if extra urgent work arrives → the 5 busy workers struggle
- The risk of missing deadlines (SLA violations) goes up

Quick Tip / Memory Trick

Consolidation trade-off:

↓ Power (good) vs ↑ SLA violation risk (bad)

This is why cloud providers must balance efficiency vs performance buffers carefully.

1.10 Q10 — Avoiding Hotspots: Load Balancing and VM Migration

What is a Hotspot?

A **hotspot** occurs when one server in a cluster becomes **overloaded with too many tasks** while other servers have very little to do (they are sitting idle).

This creates an imbalance that causes:

- Slow performance for users whose requests land on the overloaded server
- Wasted capacity on the idle servers
- Potential crashes if the hotspot server runs out of resources

The Shop Queue Analogy

Imagine a shopping mall with 4 billing counters:

- Counter 1: 50 people queuing (overwhelmed)
- Counters 2, 3, 4: Empty (completely free)

This is a hotspot. The solution is to redirect some customers from Counter 1 to the empty counters.

► Two Techniques to Avoid Hotspots

Technique 1 — Load Balancing:

Distribute incoming requests or tasks evenly across all servers so no single server gets overwhelmed.

- A **load balancer** acts like a traffic cop, deciding which server handles each request
- Works well for new incoming requests

Technique 2 — VM Migration (Live Migration):

If a server is already overloaded, **move some of its running Virtual Machines** to less-busy servers — without shutting them down.

- The VM continues running during the move (live migration)
- The overloaded server is relieved; the receiving server takes up the slack
- Like redirecting restaurant customers to a nearby less-busy branch

Correct Answer: VM Migration and Load Balancing prevent hotspots

- **Load balancing** prevents hotspots from forming (proactive)
- **VM migration** fixes hotspots that already exist (reactive)
- Together, they ensure **even resource utilization** across the cluster

Quick Tip / Memory Trick

Quick Recap — Part A Memory Tricks:

Q1: Less runtime $\Rightarrow$ 4hr downtime hits harder $\Rightarrow$ SLA violation when $Y < 20$

Q2: More cores ON = more power RIGHT NOW

Q3: Hybrid cloud = less CapEx (hardware) + more OpEx (cloud bills)

Q4: Elasticity = auto-scale up/down = pay only for what you use

- Q5: Multiple OS on one machine = Virtualization (Hypervisor)
- Q6: Don't move data to code; move code to data = Data Locality
- Q7: Bigger HDFS blocks = fewer mappers = less overhead
- Q8: Containers = lighter + higher density but weaker isolation than VMs
- Q9: Fewer servers = less power BUT more SLA risk
- Q10: Overloaded server = hotspot; fix with load balancing + VM migration

Part B — Cloud Security MCQ Concepts

Security Concepts Overview

Security in cloud computing revolves around protecting four core properties of information:

- **Confidentiality:** Information is only accessible to authorised people (secrets stay secret)
- **Integrity:** Information is not modified without permission (data stays accurate)
- **Availability:** Systems and data are accessible when needed (service stays up)
- **Authenticity:** You can verify who you are really talking to (identity is genuine)
- **Non-repudiation:** You cannot deny an action you performed (proof of actions)

2.1 Q1 — Replay Attack and Authenticity

The Story: Copying a Key Card

Imagine you have a digital key card to enter your office building. Every morning you scan it at the door:

- You scan → door opens (system generates a **session token**: a temporary digital ID)
- A spy secretly **copies your token** while you're using it
- Next day, the spy uses your **copied token** at the door
- The door opens — the building thinks the spy is you!

This is a **Replay Attack**: the attacker “replays” a valid, previously captured authentication token.

► What Security Property is Violated?

The problem here is **Authenticity**:

Correct Answer: Authenticity is violated

Authenticity = the ability to verify that someone is **genuinely who they claim to be**.

In a replay attack:

- The system cannot correctly verify identity
- It believes the attacker is the legitimate user
- The attacker has not broken the data (not an integrity issue)
- The attacker has not read secrets (not primarily a confidentiality issue)
- The core damage is **fake identity being accepted**

Quick Tip / Memory Trick

Prevention: Use **timestamps** or **one-time tokens** (nonces) in authentication. If a token has already been used or is too old, reject it — even if it's valid.

2.2 Q2 — Phishing Attack and Confidentiality

The Story: The Fake Bank Website

You receive an email that looks exactly like it's from SBI Bank. Professional logo, correct colours, familiar language. It says:

"Your account has been flagged for suspicious activity! Click here immediately to verify your account."

You click. The website looks exactly like SBI's real site. You enter:

- Your username and password
- Maybe your card number and OTP

But this website was **completely fake** — built by an attacker to steal your credentials. Now the attacker has all your private login information.

Correct Answer: Phishing attacks violate Confidentiality

Confidentiality = private information remains secret and accessible only to authorised people.

In phishing:

- Your **secret credentials** (passwords, card details) are now known to an unauthorised person
- The privacy of your sensitive information is **completely broken**
- This is a classic confidentiality breach: secrets are stolen

Quick Tip / Memory Trick

Simple Formula: Phishing = stealing secrets = Confidentiality violated.

How to remember: Phishing *fishes* for your private information.

Prevention: Always check the website URL carefully. Real banks never ask for passwords via email.

2.3 Q3 — Passive Attack: Traffic Analysis

Two Types of Attacks

- **Active Attack:** The attacker **modifies or disrupts** something. Examples: changing a message, injecting fake data, deleting files.
- **Passive Attack:** The attacker **only observes silently**. Nothing is changed. Examples: eavesdropping, monitoring, watching.

► Traffic Analysis — What is It?

Even if data is **encrypted** (so the attacker cannot read the actual messages), an attacker can still:

- Watch **who** is communicating with whom
- Notice **when** they communicate and **how often**

- See **how much** data is being transferred
- Detect **patterns** (e.g., heavy communication before a big event)

This information alone can be very revealing even without reading the content.

Correct Answer: Traffic Analysis is a Passive Attack

Why passive?

- No data is modified or deleted
- No system is disrupted or crashed
- The attacker is purely observing communication patterns
- Passive = silently watching without touching anything

Real-Life Analogy

Like someone standing outside your house and noting:

- What time you leave and return
- How many people visit you
- When packages are delivered

They haven't entered your house (no active intrusion), but they've learned a lot about you just by watching.

2.4 Q4 — Slow HTTP Attack and Disruption

The Restaurant Sabotage Story

A popular restaurant has 10 tables. It operates on a first-come, first-served basis.

A group of saboteurs arrives and:

- **Occupies all 10 tables**
- Orders food extremely slowly (asks questions, takes 30 minutes to decide)
- **Stays for hours** without leaving or finishing
- Real, paying customers cannot get a table and leave

The restaurant is **not broken** — staff are working, kitchen is running — but it's **effectively unusable** for legitimate customers.

► The Slow HTTP Attack

A web server can only handle a **limited number of simultaneous connections**. In a Slow HTTP attack:

- The attacker opens **many connections** to the server
- Sends HTTP headers or data **extremely slowly** (just enough to keep the connection alive)
- The server **waits patiently** for each request to complete
- These connections tie up all server slots
- **Real users cannot connect** — the server is busy waiting for the slow attacker

Correct Answer: Slow HTTP Attack causes Disruption (similar to Denial of Service)

Disruption = stopping or degrading normal service so legitimate users cannot access it.

The server is not hacked. No data is stolen. No files are modified. But the service becomes **completely unusable** — that is disruption. This is a variant of a **DoS (Denial of Service)** attack.

Quick Tip / Memory Trick

Key Idea: The attacker doesn't need to break the server. They just need to **tie up all its connection slots** so no real user can get through. Prevention: set request timeout limits on servers.

2.5 Q5 — Penetration Testing

What is Penetration Testing?

Penetration Testing (also called **ethical hacking** or **pen testing**) is when a company **deliberately hires security experts** to try to attack and break into their own systems — with full permission — to find vulnerabilities before real attackers do.

The goal is: *“Attack yourself first so you know where you’re weak.”*

► What Makes Penetration Testing Unique?

Technique	What It Does
Reconnaissance	Gather information about the target (addresses, open ports) — no attack yet
Vulnerability Scanning	Use tools to find weaknesses — identifies problems but does NOT exploit them
Penetration Testing	Actually exploits the vulnerabilities to prove they are real and show the damage

Correct Answer: Penetration Testing

Key characteristics:

- It **actually breaks in** (with permission) — doesn’t just find potential weaknesses
- It shows **real-world impact**: what data could be accessed, what systems could be controlled
- It **requires explicit permission** — because it may affect live systems
- It is used to **prove the risk is real**, not just theoretical

Real-Life Analogy

A bank hires a security consultant who says:

- Reconnaissance: “I’ll observe the building layout from outside”
- Vulnerability Scan: “I found the back door lock is weak”
- Penetration Test: “I actually unlocked the back door and walked into the vault — here’s proof it can be done”

Only penetration testing *actually proves* the vulnerability is exploitable.

2.6 Q6 — Non-repudiation and Digital Signatures

What is Non-repudiation?

Non-repudiation means you **cannot deny** that you performed a certain action, because there is strong, verifiable proof that it was you.

Example: You send money online. Later you claim: “I never sent that!” Non-repudiation means the system has **irrefutable proof** that you did send it.

► How is Non-repudiation Achieved?

The answer is **Digital Signatures**.

How a Digital Signature Works

Think of a handwritten signature — but digital and impossible to forge:

1. You have two special mathematical keys:
 - **Private key:** Only you have this. You keep it completely secret.
 - **Public key:** Everyone can see this. It's openly distributed.
2. When you perform a transaction, you **sign it with your private key**
3. Anyone can verify the signature using your **public key**
4. **Only you** could have created that signature (only you have the private key)
5. Therefore, **you cannot deny the transaction** — the proof is cryptographically sound

Correct Answer: Digital Signature ensures Non-repudiation

Why Other Options Are Wrong

- **Password:** Can be shared, stolen, or given to someone else. Not proof of *who* acted.
- **Encryption:** Hides data content, but doesn't prove who sent it. Ensures confidentiality, not identity.
- **Log files:** Can potentially be manipulated by a system administrator. Not irrefutable proof.

Only a digital signature provides mathematically verifiable, unforgeable proof of identity.

Quick Tip / Memory Trick

Non-repudiation = You did it and you cannot deny it.

Tool = Digital Signature (created with your unique private key — only you could have signed it).

2.7 Q7 — Authentication vs Authorization

Two Completely Different Things

These two terms are often confused. They sound similar but address **completely different questions**.

► Authentication — Proving Your Identity

Authentication answers: “*Who are you? Are you really who you claim to be?*”

Instagram Login Example

You open Instagram and enter your username and password. Instagram checks:

- Does this username exist?
- Does the password match what we have stored?
- If both match → Identity verified → **Authentication successful**

► Authorization — Deciding What You Can Do

Authorization answers: “*What are you allowed to do? What permissions do you have?*”

After Instagram Login

After successful authentication (login), what can you do?

- Edit your own profile? ✓ (you are authorised)
- Delete someone else’s post? × (you are NOT authorised)
- View a private account’s photos without following? × (not authorised)

These access control decisions are **Authorization**.

Concept	Question It Answers	Example
Authentication	Who are you?	Login with password
Authorization	What can you do?	Access control / permissions
Non-repudiation	Can you deny what you did?	Digital signature

Correct Answer: Statement I is False, Statement II is True

- **Statement I:** “Authorization = verifying identity” → **FALSE**. Authorization is about permissions, not identity. Authentication verifies identity.
- **Statement II:** “Non-repudiation = cannot deny action” → **TRUE**. This is the exact definition.

Quick Tip / Memory Trick

Sequence: Authentication always comes first, then Authorization.
First prove *who you are*, then the system decides *what you can do*.

2.8 Q8 — PaaS: The Middle Ground

Review: The Three Service Models

- **IaaS** — You get raw virtual machines. You manage: OS, runtime, middleware, apps. Maximum work for you.
- **PaaS** — You get a complete development platform. You manage: only your code. Provider manages: everything else.
- **SaaS** — You just use a finished application. You manage: nothing technical.

► Why PaaS is the Right Choice for Developers

Cooking Analogy

You want to cook a meal:

- **IaaS = Raw ingredients** — You buy flour, eggs, butter, vegetables. Cook everything from scratch. Maximum effort, maximum control.
- **PaaS = Equipped kitchen with semi-prepared ingredients** — Vegetables already washed and chopped, kitchen tools are ready. You just cook the meal. Focus on the recipe (your code), not the setup.
- **SaaS = Ready-made food delivery** — Open the app, order, eat. You didn't cook anything.

► What PaaS Specifically Provides

- **You upload your code** (Python, Java, Node.js, etc.)
- Provider automatically handles:
 - Operating system updates and patches
 - Runtime environment (Java JVM, Python interpreter, Node.js, etc.)
 - Middleware and web server configuration
 - Load balancing and scaling
- **You control:** Application logic, configuration settings, scaling parameters

Correct Answer: PaaS

PaaS provides developers a platform to deploy applications without managing underlying infrastructure. The developer writes code and the platform handles everything else — making it the optimal choice when the question asks about “upload code, provider manages runtime and infrastructure.”

Quick Tip / Memory Trick

PaaS Decision Rule: If you write code and want someone else to manage the servers, OS, and runtime — that's PaaS.
Examples: Heroku, Google App Engine, AWS Elastic Beanstalk, Firebase.

2.9 Q9 — Data Location / Data Residency

The Core Concept

Data Location (also called **Data Residency**) refers to the **physical geographic location** of the servers and storage where your data is actually kept.

This matters because **different countries have different laws** about how data must be stored, protected, and who can access it.

► Why This is a Cloud Challenge

The Legal Compliance Problem

Imagine a hospital in India:

- It moves patient records to a cloud provider
- The cloud provider's servers are physically located in the United States
- Indian law requires: *patient health data must remain within India's borders*
- Even if the data is completely encrypted and secure, this is still **a legal violation**
- The issue is not security — the issue is **where the data physically lives**

► Different Countries, Different Rules

- **European Union (GDPR):** Personal data of EU citizens must generally be stored or protected in the EU
- **India:** Financial and health data must reside on servers in India in many cases
- **China:** Government data cannot leave Chinese borders

Correct Answer: Data Location / Data Residency

The concern is specifically about **where data is physically stored** — not about general security, encryption strength, or operational procedures. Even with perfect security, violating data residency laws creates serious legal and compliance problems for organisations.

Quick Tip / Memory Trick

Data Location = not *how secure* is the data, but *where in the world* is the data sitting right now?

Cloud providers often offer region-specific data centres (Mumbai, Singapore, Frankfurt) to help customers satisfy data residency requirements.

2.10 Q10 — Vendor Lock-in

What is Vendor Lock-in?

Vendor Lock-in is a situation where a customer becomes so deeply dependent on one cloud provider's tools, formats, and APIs that switching to a different provider becomes extremely difficult, costly, and risky.

► How Vendor Lock-in Happens

Building Everything on One Platform

Your startup builds an application completely on **Amazon Web Services (AWS)**:

- Database: Amazon DynamoDB (AWS-specific format)
- Functions: AWS Lambda (AWS-specific serverless)
- Storage: Amazon S3 (widely used but proprietary API)
- Messaging: Amazon SQS (AWS-specific queue service)
- Machine learning: Amazon SageMaker (AWS-specific ML platform)

Now your company wants to move to Google Cloud Platform (GCP) to save costs.

Problems:

- DynamoDB data format doesn't directly transfer to Google's Firestore
- Lambda functions need to be rewritten for Google Cloud Functions
- All the AWS-specific code must be refactored
- Migration could take months and cost millions
- During migration, service may be disrupted

Correct Answer: Vendor Lock-in — difficulty in migrating data and applications

The **main problem** is:

- Migrating data to another provider is technically complex
- Application code tied to proprietary APIs must be rewritten
- High cost and time required to switch
- **Not** about data theft, misuse, or security breaches

Why Other Options Are Wrong

- **Not about data theft** — data is safe; it just can't easily move
- **Not about data misuse** — the provider isn't abusing your data
- **Not about security audits** — audit compliance is a different concern
- The issue is purely **switching difficulty and dependency**

Quick Tip / Memory Trick

How to avoid vendor lock-in?

- Use **open standards** and **portable formats** wherever possible
- Design applications using **cloud-agnostic architectures**
- Use **containers (Docker/Kubernetes)** which can run on any cloud

- Avoid relying entirely on provider-specific proprietary services

Master Revision Cheat Sheet — All 20 Questions

3.1 Part A: Cloud Computing

Q	Topic	Key Concept	Answer / Trick
1	SLA Availability	$\text{Availability} = 1 - \frac{\text{down-time}}{\text{total}}$	$Y < 20$ causes SLA violation
2	Multi-core Power	More active cores = more power	Instantaneous power increases
3	CapEx vs OpEx	Hybrid cloud shifts spending	CapEx ↓ OpEx ↑
4	Elastic Scalability	Auto scale up/down with demand	Pay only for actual usage
5	Virtualization	Hypervisor creates VMs	Multiple OS on one machine
6	MapReduce Locality	Move code to data, not data to code	Schedule mappers near data
7	HDFS Block Size	Bigger blocks = fewer mappers	Less scheduling overhead
8	Containers vs VMs	Containers share OS kernel	Higher density, weaker isolation
9	Energy Consolidation	Fewer servers = less power	SLA violation risk increases
10	Hotspots	One server overloaded, others idle	Load balancing + VM migration

3.2 Part B: Cloud Security

Q	Topic	Key Concept	Answer / Trick
1	Replay Attack	Stolen token replayed to fake identity	Authenticity is violated
2	Phishing	Fake site steals credentials	Confidentiality is violated
3	Passive Attack	Watching without touching	Traffic analysis = passive
4	Slow HTTP	Tie up server connections	Disruption / DoS attack
5	Penetration Testing	Actually exploit vulnerabilities	Ethical hacking with permission
6	Non-repudiation	Cannot deny your own action	Digital Signature (private key)
7	Auth vs AuthZ	Identity vs Permissions	Auth = who; AuthZ = what
8	PaaS	Upload code, provider manages rest	Heroku, Google App Engine
9	Data Location	Where data physically lives	Country law compliance
10	Vendor Lock-in	Dependent on one provider	Hard to migrate = lock-in

Ultimate Memory Summary

Cloud: SLA math | Cores use power | CapEx→OpEx | Elastic = pay as you use | VMs = multiple OS

Data Locality | Big blocks = less overhead | Containers = light but shared kernel
Consolidation = save power but risk SLA | Hotspot fix = load balance + migrate VMs

Security: Replay = fake identity = Authenticity | Phishing = steal secrets = Confidentiality

Traffic watch = Passive | Slow HTTP = Disruption | Pen test = ethical hack (exploit + prove)

Non-repudiation = Digital Signature | Auth = identity; AuthZ = permission

PaaS = dev uploads code | Data Location = where servers are | Lock-in = can't switch easily